

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	Y. Jayesh Ranjan	Reg. No.:	192472201
Section / Batch:	CSE - AI	Date:	16/08/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I Y. Jayesh Ranjan - 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

15. Prompt-Based Python Code Generator: Accept a programming problem in natural language and generate Python code using an LLM API. Include at least two test inputs.

26. Basic RAG Question Answering System: Develop a simple RAG application using Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM. Answer questions based on supplied documents.

Q15 — Prompt-Based Python Code Generator

Aim

To develop a prompt-based Python code generator using the Gemini API that generates Python programs from a given programming problem and tests the generated code using two test inputs.

Algorithm

1. Start the program.
 2. Accept a programming problem from the user.
 3. Send the problem to the Gemini API.
 4. Generate Python code using Gemini.
 5. Display the generated Python code.
 6. Accept two test inputs from the user.
 7. Execute the generated Python code with each test input.
 8. Display the outputs for both test cases.
 9. Stop the program.
-

Code

```
import subprocess

    timeout=10
)

    if result.returncode != 0:
        return f"Program Error:\n{result.stderr.strip()}"

    output = result.stdout.strip()

    if not output:
        return "(Program produced no output.)"

    return output

except subprocess.TimeoutExpired:
    return "Program Error: Execution timed out."

finally:
    if os.path.exists(temp_file_path):
        os.remove(temp_file_path)
```

```

def main():
    print("=" * 60)
    print("    PROMPT-BASED PYTHON CODE GENERATOR")
    print("=" * 60)

    problem = input("\nEnter your programming problem:\n> ").strip()

    if not problem:
        print("\nError: Programming problem cannot be empty.")
        return

    print("\nGenerating Python code using Gemini...")

    try:
        generated_code = generate_python_code(problem)
        print("\nGENERATED PYTHON CODE")
        print("=" * 60)
        print(generated_code)

    except Exception as error:
        print("\nError while generating code:")
        print(error)
        return

    test_input_1 = input("\nEnter Test Input 1:\n> ").strip()
    test_input_2 = input("\nEnter Test Input 2:\n> ").strip()

    if not validate_test_inputs(test_input_1, test_input_2):
        print("\nError: Both test inputs are required.")
        return

    print("\nTEST RESULTS")
    print("=" * 60)

```

```

result_1 = run_generated_code(
    generated_code,
    test_input_1
)

print("\nTest Input 1:", test_input_1)
print("Output 1:", result_1)

result_2 = run_generated_code(
    generated_code,
    test_input_2
)

print("\nTest Input 2:", test_input_2)
print("Output 2:", result_2)

if __name__ == "__main__":
    main()

```

Input

Programming problem:

Write a Python program to check whether a number is prime.

Test Input 1:

7

Test Input 2:

13

Output

PROMPT-BASED PYTHON CODE GENERATOR

Generating Python code using Gemini...

GENERATED PYTHON CODE

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False  
    return True
```

...

Test Input 1: 7

Output 1: 7 is a prime number.

Test Input 2: 13

Output 2: 13 is a prime number.

```

> program to check a number is prime or not

Generating Python code using Gemini...

=====
GENERATED PYTHON CODE
=====
import math

def is_prime(n: int) -> bool:
    """Returns True if n is a prime number, False otherwise."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    # Check odd factors up to the square root of n
    limit = math.isqrt(n)
    for i in range(3, limit + 1, 2):
        if n % i == 0:
            return False
    return True

def main():
    try:
        num = int(input("Enter an integer to check: "))
        if is_prime(num):
            print(f"{num} is a prime number.")
        else:
            print(f"{num} is not a prime number.")
    except ValueError:
        print("Please enter a valid integer.")

if __name__ == "__main__":
    main()
=====
TWO TEST INPUTS
=====

Enter Test Input 1:
> 18

Enter Test Input 2:
> 2

=====
TEST RESULTS
=====

Test Input 1:
18

Output 1:
Enter an integer to check: 18 is not a prime number.

-----

Test Input 2:
2

Output 2:
Enter an integer to check: 2 is a prime number.
(.venv) PS C:\Users\JAYESH\OneDrive\Desktop\API_ASSIGNMENT>

```

Result

Thus, the prompt-based Python code generator was successfully implemented using the Gemini API, and the generated program was successfully tested with two inputs.

Q26 — Basic RAG Question Answering System

Aim

To develop a basic Retrieval-Augmented Generation (RAG) system that retrieves relevant information from documents using embeddings and ChromaDB and generates an answer using Gemini.

Algorithm

1. Start the program.
 2. Load the documents from the documents folder.
 3. Split the documents into smaller chunks.
 4. Generate embeddings for the document chunks using Gemini.
 5. Store the embeddings in ChromaDB.
 6. Accept a question from the user.
 7. Generate an embedding for the question.
 8. Retrieve the most relevant document chunks using similarity search.
 9. Send the question and retrieved context to Gemini.
 10. Generate a document-grounded answer.
 11. Display the retrieved chunks and generated answer.
 12. Stop the program.
-

Code

```
from .document_loader import load_documents
from .embeddings import generate_embeddings
from .vector_store import (
    create_vector_store,
    add_chunks_to_vector_store,
)
from .retriever import retrieve_relevant_chunks
from .rag import generate_rag_answer

DOCUMENT_PATH = "Q26_Basic_RAG/documents"

def build_knowledge_base():
    documents = load_documents(DOCUMENT_PATH)

    chunks = chunk_documents(documents)
```

```
embeddings = generate_embeddings(  
    [chunk["text"] for chunk in chunks]  
)
```

```
collection = create_vector_store()
```

```
add_chunks_to_vector_store(  
    collection,  
    chunks,  
    embeddings,  
)
```

```
return collection
```

```
def main():  
    print("=" * 60)  
    print("          BASIC RAG QUESTION ANSWERING")  
    print("=" * 60)
```

```
collection = build_knowledge_base()
```

```
question = input(  
    "\nEnter your question:\n> "  
)
```

```
if not question:  
    print("Error: Question cannot be empty.")  
    return
```

```
print("\nRetrieving relevant documents...")
```

```
retrieved_chunks = retrieve_relevant_chunks(  
    collection,
```



```
    question,  
    top_k=3,  
)
```

```
print("\nRETRIEVED DOCUMENT CHUNKS")  
print("=" * 60)
```

```
for index, chunk in enumerate(  
    retrieved_chunks,  
    start=1  
):  
    print(f"\n[ {index}] Source: {chunk['source']}")  
    print(f"Text:\n{chunk['text']}")
```

```
print("\nGENERATING ANSWER...")  
print("=" * 60)
```

```
answer = generate_rag_answer(  
    question,  
    retrieved_chunks  
)
```

```
print("\nGENERATED ANSWER")  
print("=" * 60)  
print(answer)
```

```
if __name__ == "__main__":  
    main()
```

Input

Question:

What are the attendance regulations?

Output

RETRIEVED DOCUMENT CHUNKS

[1] Source: attendance.txt

Attendance Regulations

Students are required to maintain the minimum attendance percentage specified by the college for their courses.

Attendance is recorded for lectures, practical sessions, and other scheduled academic activities.

...

GENERATED ANSWER

Based on the provided documents, the attendance regulations are as follows:

- Students must maintain the minimum attendance percentage specified by the college.
- Attendance is recorded for lectures, practical sessions, and other academic activities.
- Students should regularly attend classes and monitor their attendance.
- Attendance-related issues should be reported to the appropriate faculty member or academic department.

```

(.venv) PS C:\Users\JAYESH\OneDrive\Desktop\API_ASSIGNMENT> python -m q2o_basic_rag.main
=====
BASIC RAG QUESTION ANSWERING
=====
Loading documents...
Documents loaded: 3
Chunks created: 4
Embeddings generated: 4
Vector database records: 4

=====
Enter your question:
> what is the mininum attendance percentage

Retrieving relevant documents...

=====
RETRIEVED DOCUMENT CHUNKS
=====

[1] Source: attendance.txt
Chunk: 0
Distance: 0.5146
Text:
Attendance Regulations

=====
GENERATING ANSWER...
=====

GENERATED ANSWER
=====
Based on the supplied documents, the exact minimum attendance percentage is not specified. The documents only state that st
udents are required to maintain the minimum attendance percentage "specified by the college for their courses." Therefore,
the specific percentage information is not available in the supplied documents.
(.venv) PS C:\Users\JAYESH\OneDrive\Desktop\API_ASSIGNMENT>

```

Result

Thus, the basic RAG question-answering system was successfully implemented using Gemini embeddings, ChromaDB, and Gemini, and relevant document-based answers were successfully generated.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						